

Assignment 4: Tool Specification

Personal Productivity and Task Execution Agent

This document specifies all 14 tools implemented by the agent (8 required + 6 bonus/extra, including two — `delete_task` and `list_reminders` — added after the original 12-tool version of this spec) in enough detail that another developer could implement each tool independently, without reading the project's source code. Every field name, type, default, error condition, and example below matches the actual running implementation (`app/tools/*.py`), including the code-level rule that force-gates multi-task creation behind approval (see Tool 1).

Tool 1: `create_task`

Tool Name	<code>create_task</code>
Purpose	Create a single new task for the user to track (title, optional description, priority, due date, and tags).
Input Schema	• <code>title</code> — string — the task's short title • <code>description</code> — string — longer free-text description (default: empty string) • <code>priority</code> — string enum: Low Medium High Critical (default: Medium) • <code>due_date</code> — string — ISO date, format YYYY-MM-DD (optional) • <code>tags</code> — array of string (default: empty array)
Required Fields	Title
Optional Fields	description, priority, due_date, tags
Output Schema	• <code>task</code> — object — the full created Task record (<code>task_id</code>, title, description, priority, status, due_date, created_date, updated_date, tags, source, notes) • <code>task_id</code> — string — unique ID of the created task, format TASK-xxxxxxx • <code>confirmation_status</code> — string — always "created" on success
Read or Write	Write
Approval Required	No for the 1st <code>create_task</code> call in a user turn. YES for the 2nd and every subsequent <code>create_task</code> call within the SAME turn — Requirement 7 mandates approval before "creating multiple tasks", enforced in code by the Agent Controller (a per-turn counter), not left to the model's judgement alone.

Possible Errors	- Invalid priority value not in the enum → tool input validation error ("Invalid arguments for tool 'create_task': ...") - Invalid due_date not in YYYY-MM-DD format → tool input validation error - Missing required 'title' → tool input validation error ("field required")
Example Call	<pre>{"title": "Buy groceries", "priority": "High", "due_date": "2026-07-18", "tags": ["errands"]}</pre>
Example Result	<pre>{ "task": { "task_id": "TASK-69e1c864", "title": "Buy groceries", "description": "", "priority": "High", "status": "Pending", "due_date": "2026-07-18", "created_date": "2026-07-16T09:30:00Z", "updated_date": "2026-07-16T09:30:00Z", "tags": ["errands"], "source": "agent", "notes": "" }, "task_id": "TASK-69e1c864", "confirmation_status": "created" }</pre>

Tool 2: list_tasks

Tool Name	list_tasks
Purpose	List and filter existing tasks by status, priority, a due-date range, and/or a tag.
Input Schema	- status — string enum: Pending In Progress Blocked Completed Cancelled (optional) - priority — string enum: Low Medium High Critical (optional) - due_before — string — ISO date, inclusive upper bound (optional) - due_after — string — ISO date, inclusive lower bound (optional) - tag — string (optional)
Required Fields	(none — all fields optional)
Optional Fields	status, priority, due_before, due_after, tag

Output Schema	• tasks — array of Task objects matching the filters • total_count — integer — number of matching tasks
Read or Write	Read
Approval Required	No
Possible Errors	• status/priority value not in the enum → tool input validation error • No matching tasks → not an error; returns tasks: [] and total_count: 0
Example Call	<pre>{"priority": "High", "status": "Pending"}</pre>
Example Result	<pre>{ "tasks": [{ "task_id": "TASK-69e1c864", "title": "Buy groceries", "priority": "High", "status": "Pending", "due_date": "2026-07-18", "tags": ["errands"], "...": "..." }], "total_count": 1 }</pre>

Tool 3: update_task

Tool Name	update_task
Purpose	Update one or more fields of an existing task, identified by task_id.
Input Schema	• task_id — string — ID of the task to update • title — string (optional) • description — string (optional) • priority — string enum: Low Medium High Critical (optional) • due_date — string — ISO date (optional) • status — string enum: Pending In Progress Blocked Completed Cancelled (optional) • tags — array of string (optional)
Required Fields	task_id
Optional Fields	title, description, priority, due_date, status, tags

Output Schema	- task — object — the updated Task record (on success) - confirmation_status — string — "updated" (on success) - error — string — present instead of the above if task_id does not exist
Read or Write	Write
Approval Required	Yes — always required before execution
Possible Errors	- Unknown task_id → {"error": "No task found with ID '<task_id>'"} - Invalid priority/status value not in the enum → tool input validation error
Example Call	<code>{"task_id": "TASK-69e1c864", "status": "In Progress"}</code>
Example Result	<pre>{ "task": {"task_id": "TASK-69e1c864", "status": "In Progress", "...": "..."}, "confirmation_status": "updated" }</pre>

Tool 4: complete_task

Tool Name	complete_task
Purpose	Mark an existing task as Completed and record a completion timestamp.
Input Schema	task_id — string — ID of the task to complete
Required Fields	task_id
Optional Fields	(none)
Output Schema	- task — object — the updated Task record (on success) - updated_status — string — "Completed" (on success) - completion_timestamp — string — ISO 8601 UTC timestamp (on success) - error — string — present instead of the above if task_id does not exist
Read or Write	Write
Approval Required	Yes — always required before execution

Possible Errors	Unknown task_id → {"error": "No task found with ID '<task_id>'"}
Example Call	<pre>{"task_id": "TASK-69e1c864"}</pre>
Example Result	<pre>{ "task": {"task_id": "TASK-69e1c864", "status": "Completed", "...": "..."}, "updated_status": "Completed", "completion_timestamp": "2026-07-16T10:15:00+00:00" }</pre>

Tool 5: search_notes

Tool Name	search_notes
Purpose	Search saved notes by keyword/topic, optionally filtered by category or a created-date range.
Input Schema	- query — string — search text - category — string (optional) - date_from — string — ISO datetime lower bound (optional) - date_to — string — ISO datetime upper bound (optional)
Required Fields	Query
Optional Fields	category, date_from, date_to
Output Schema	- notes — array of note objects, each including a match_score (float relevance ranking) - total_count — integer — number of matching notes
Read or Write	Read
Approval Required	No
Possible Errors	- Missing required 'query' → tool input validation error ("field required") - No matches → not an error; returns notes: [] and total_count: 0
Example Call	<pre>{"query": "marketing campaign"}</pre>
Example Result	<pre>{ "notes": [</pre>

	<pre> {"note_id": "NOTE-3f8a21c9", "title": "Marketing campaign notes", "content": "...", "category": "Meetings", "match_score": 0.42}], "total_count": 1 } </pre>
--	---

Tool 6: save_note

Tool Name	save_note
Purpose	Save a new note with a title, content, category, and tags.
Input Schema	• title — string • content — string • category — string (default: "General") • tags — array of string (default: empty array)
Required Fields	title, content
Optional Fields	category, tags
Output Schema	• note_id — string — unique ID of the saved note, format NOTE-xxxxxxx • confirmation_status — string — always "saved" on success
Read or Write	Write
Approval Required	No
Possible Errors	• Missing 'title' or 'content' → tool input validation error ("field required")
Example Call	<pre> {"title": "Marketing ideas", "content": "Discussed Q3 budget", "category": "Meetings"} </pre>
Example Result	<pre> {"note_id": "NOTE-3f8a21c9", "confirmation_status": "saved"} </pre>

Tool 7: extract_meeting_actions

Tool Name	extract_meeting_actions
Purpose	Analyze raw meeting notes or a transcript and extract a structured summary, decisions, action items (with owner/deadline when stated),

	and unresolved questions, via a dedicated LLM sub-call with a fixed output schema.
Input Schema	transcript — string — raw meeting notes or transcript text
Required Fields	Transcript
Optional Fields	(none)
Output Schema	- summary — string — 2-4 sentence summary - decisions — array of string - action_items — array of {description: string, owner: string null, deadline: string null} - unresolved_questions — array of string - error — string — present instead of the above on failure
Read or Write	Read (no persistence — pure text analysis via the LLM)
Approval Required	No
Possible Errors	- Empty transcript → {"error": "Meeting notes/transcript text is empty."} - LLM API failure (timeout, quota, etc.) → {"error": "Extraction failed: <reason>"} - Malformed/unparseable LLM JSON output → {"error": "Extraction returned an unexpected shape: <reason>"}
Example Call	<pre>{"transcript": "We agreed to launch by Friday. Ali will prepare the deck."}</pre>
Example Result	<pre>{ "summary": "The team agreed on a Friday launch date and assigned deck preparation.", "decisions": ["Launch by Friday"], "action_items": [{"description": "Prepare the deck", "owner": "Ali", "deadline": null}], "unresolved_questions": [] }</pre>

Tool 8: generate_work_plan

Tool Name	generate_work_plan
------------------	--------------------

Purpose	Generate an ordered daily work plan from the user's pending/in-progress tasks, considering priority, due date, status, and estimated effort.
Input Schema	• available_hours — number, $0 < \text{value} \leq 24$ — hours available today • date — string — ISO date to plan for (default: today) • user_priorities — array of string — keywords/tags to weight higher (optional)
Required Fields	available_hours
Optional Fields	date, user_priorities
Output Schema	• date — string — the ISO date the plan was generated for • ordered_schedule — array of {task_id, title, priority, due_date, estimated_hours, priority_score, reason} • recommended_focus_areas — array of string (priority levels to focus on) • deferred_tasks — array of {task_id, title, priority, due_date, reason} • risk_warnings — array of string (e.g. overdue or blocked items) • hours_allocated — number • hours_remaining — number
Read or Write	Read (reads tasks; writes nothing)
Approval Required	No
Possible Errors	• available_hours outside (0, 24] → tool input validation error • Invalid 'date' not in YYYY-MM-DD format → {"error": "'date' must be an ISO date (YYYY-MM-DD), got '<value>'"}
Example Call	<code>{"available_hours": 4, "user_priorities": ["marketing"]}</code>
Example Result	<pre>{ "date": "2026-07-17", "ordered_schedule": [{"task_id": "TASK-69e1c864", "title": "Buy groceries", "priority": "High", "due_date": "2026-07-18", "estimated_hours": 2.0, "priority_score": 5.0, "reason": "High priority, due in 1d"}], "recommended_focus_areas": ["Critical", "High"],</pre>

	<pre>"deferred_tasks": [], "risk_warnings": [], "hours_allocated": 2.0, "hours_remaining": 2.0 }</pre>
--	--

Tool 9: create_reminder

Tool Name	create_reminder
Purpose	Create a reminder for a given title and time, optionally linked to an existing task.
Input Schema	• title — string • remind_at — string — ISO date or datetime • related_task_id — string (optional)
Required Fields	title, remind_at
Optional Fields	related_task_id
Output Schema	• reminder_id — string — unique ID, format REM-xxxxxxx (on success) • confirmation_status — string — "created" (on success) • error — string — present instead of the above if related_task_id does not exist
Read or Write	Write
Approval Required	Yes — always required before execution
Possible Errors	• related_task_id does not exist → {"error": "No task found with ID '<id>'"}>
Example Call	{"title": "Project review", "remind_at": "2026-08-05"}
Example Result	{"reminder_id": "REM-7a12ef90", "confirmation_status": "created"}

Tool 10: draft_follow_up_email

Tool Name	draft_follow_up_email
Purpose	Draft (never send) a follow-up email based on meeting notes, returning a subject and body the user can review, edit, and copy.
Input Schema	• meeting_notes — string

Required Fields	- recipient_name — string (optional) - tone — string (default: "professional")
	meeting_notes
	recipient_name, tone
Optional Fields	
Output Schema	- subject — string - body — string - tone — string — echoes the requested tone - status — string — always "draft_only_not_sent" - error — string — present instead of the above if meeting_notes is empty
Read or Write	Read (no persistence — generates a draft only)
Approval Required	Yes — Requirement 7 lists "sending or simulating an email" as approval-required; a drafted email is gated even though it is never actually sent.
Possible Errors	Empty meeting_notes → {"error": "meeting_notes is empty."}
Example Call	<pre>{"meeting_notes": "We agreed to launch Friday.", "recipient_name": "Ali"}</pre>
Example Result	<pre>{ "subject": "Follow-up: Meeting Recap & Next Steps", "body": "Hi Ali,\n\nThanks for the discussion earlier...", "tone": "professional", "status": "draft_only_not_sent" }</pre>

Tool 11: detect_overdue_tasks

Tool Name	detect_overdue_tasks
Purpose	Find all non-completed tasks whose due_date is before a given (or the current) date, ranked by priority and days overdue.
Input Schema	as_of_date — string — ISO date (default: today)
Required Fields	(none — all fields optional)
Optional Fields	as_of_date

Output Schema	- overdue_tasks — array of Task objects, each with an added days_overdue integer - total_count — integer - as_of_date — string — the ISO date used as the reference point
Read or Write	Read
Approval Required	No
Possible Errors	Invalid as_of_date not in YYYY-MM-DD format → {"error": "'as_of_date' must be ISO date, got '<value>'"} }
Example Call	<code>{}</code>
Example Result	<pre>{ "overdue_tasks": [{"task_id": "TASK-d4ee8ed4", "title": "Call client", "days_overdue": 3, "...": "..."}], "total_count": 1, "as_of_date": "2026-07-17" }</pre>

Tool 12: generate_weekly_report

Tool Name	generate_weekly_report
Purpose	Generate a weekly productivity report: tasks completed and created this week, overdue and blocked tasks, and recommended priorities for next week.
Input Schema	week_ending — string — ISO date (default: today)
Required Fields	(none — all fields optional)
Optional Fields	week_ending
Output Schema	- week_start, week_ending — string, ISO dates bounding the 7-day window - completed_count, created_count, overdue_count, blocked_count — integers - completed_tasks, overdue_tasks, blocked_tasks — arrays of Task objects - recommended_priorities_next_week — array of {task_id, title, priority, due_date}

Read or Write	Read
Approval Required	No
Possible Errors	Invalid week_ending not in YYYY-MM-DD format → {"error": "'week_ending' must be ISO date, got '<value>'"}
Example Call	<code>{}</code>
Example Result	<pre>{ "week_start": "2026-07-10", "week_ending": "2026-07-17", "completed_count": 2, "created_count": 3, "overdue_count": 1, "blocked_count": 0, "completed_tasks": [...], "overdue_tasks": [...], "blocked_tasks": [], "recommended_priorities_next_week": [{"task_id": "TASK-91147fce", "title": "Submit assignment", "priority": "High", "due_date": "2026-07-18"}] }</pre>

Tool 13: delete_task

Tool Name	delete_task
Purpose	Permanently delete an existing task, identified by task_id. This is a destructive, irreversible action — the underlying row is removed from the database rather than soft-deleted (a soft delete is already available via update_task's status field, e.g. Cancelled).
Input Schema	task_id — string — ID of the task to delete
Required Fields	task_id
Optional Fields	(none)
Output Schema	- task_id — string — the ID of the task that was deleted (on success) - deleted_task — object — the full Task record as it existed immediately before deletion (on success) - confirmation_status — string — always "deleted" (on success) - error — string — present instead of the above if task_id does not exist
Read or Write	Write (destructive)

Approval Required	Yes — always required before execution, unconditionally (same gating as <code>update_task</code> and <code>complete_task</code> ; there is no code path that can execute <code>delete_task</code> without approval).
Possible Errors	Unknown <code>task_id</code> → <code>{"error": "No task found with ID '<task_id>'"} </code>
Example Call	<code>{"task_id": "TASK-69e1c864"}</code>
Example Result	<pre>{ "task_id": "TASK-69e1c864", "deleted_task": {"task_id": "TASK-69e1c864", "title": "Buy groceries", "...": "..."}, "confirmation_status": "deleted" }</pre>

Tool 14: `list_reminders`

Tool Name	<code>list_reminders</code>
Purpose	List every reminder the user has created, sorted soonest (earliest <code>remind_at</code>) first. Backs any request to see, count, or review reminders — the same data shown in the app's Reminders panel.
Input Schema	(none — this tool takes no arguments)
Required Fields	(none — takes no arguments)
Optional Fields	(none)
Output Schema	<code>reminders</code> — array of Reminder objects (<code>reminder_id</code>, <code>title</code>, <code>remind_at</code>, <code>related_task_id</code>, <code>created_date</code>), soonest first <code>total_count</code> — integer — number of reminders returned
Read or Write	Read
Approval Required	No
Possible Errors	None — always returns a list (empty if no reminders exist yet); this tool cannot error under normal operation
Example Call	<code>{}</code>
Example Result	<pre>{ "reminders": [</pre>

```
{
  "reminder_id": "REM-69e1c864", "title": "Call the dentist",
  "remind_at": "2026-07-21T09:00:00Z", "related_task_id": null,
  "created_date": "2026-07-16T09:30:00Z"
},
{
  "total_count": 1
}
```